
Complete Technical Report

Network Traffic Feature Analysis
& Development Roadmap

Domain: Cybersecurity & Network Analysis &
Advanced algorithms

</> Technologies: Python, NetworkX,
Scikit-learn

📊 Goal: Automated threat detection

Year: 2025–2026

Contents

1	Introduction	2
1.1	Context and Objectives	2
2	Network Traffic Features	3
2.1	Timestamp	3
2.2	 Duration	3
2.3	 Protocol	4
2.4	 SourceIP	4
2.5	 DestinationIP	5
2.6	 SourcePort	5
2.7	 DestinationPort	5
2.8	 PacketCount	6
2.9	 BytesSent	6
2.10	 BytesReceived	7
2.11	 Label	7
2.12	 Features Summary Table	8
3	Development Roadmap	9
4	Conclusion	13

1 Introduction

This report presents the **NIDS** (Network Intrusion Detection System) project — a complete network intrusion detection system based on graph analysis and machine learning. It covers two complementary parts: detailed description of the **network traffic features** analyzed, and the **development roadmap** structured in five progressive phases.

1.1 Context and Objectives

Network security is a critical challenge. Traditional intrusion detection systems rely on static signatures, which are insufficient against modern threats and zero-day attacks. This project proposes a hybrid approach combining:

- **Graph analysis** to model relationships between hosts;
- **Isolation Forest** to detect behavioral anomalies;
- A **contextual rule engine** to generate explainable alerts;
- An **interactive dashboard** for real-time visualization.

2 Network Traffic Features

i The input CSV file contains the following columns. Each row represents a recorded network connection.

2.1 Timestamp

Timestamp

Definition: Date and time of connection start or recording.

Example:

```
2024-03-15 14:23:05
```

Role in the system:

- Enables temporal analysis of events.
- Detection of recurring attacks at specific times (night scans).
- Correlation of events across multiple logs.
- Used for **traffic spike detection** (60s sliding window).

2.2 Duration

Duration

Definition: Connection duration, expressed in seconds.

Example:

```
24.0777 -> 24 seconds connection
```

Role in the system:

- Very short connections with many packets → scanning.
- Very long connections → persistent data exfiltration.
- Key feature for Isolation Forest.

2.3 Protocol

Protocol

Definition: Communication protocol used for the connection.

Possible values:

Protocol	Type	Typical usage
TCP	Connection-oriented, reliable	Web, SSH, FTP
UDP	Connectionless, fast	DNS, streaming
ICMP	Control messages	Ping, traceroute

Role in the system: Certain attacks exploit specific protocols (ICMP floods, TCP SYN scans, DNS tunneling via UDP).

2.4 SourceIP

SourceIP

Definition: IP address of the machine **initiating** the connection.

Example:

192.168.1.239

Role in the system:

- Source node in the NetworkX graph.
- Basis for per-IP feature extraction (out-degree, unique neighbors).
- Computation of `out_degree` and `betweenness centrality`.

2.5 DestinationIP

DestinationIP

Definition: IP address of the **destination** machine.

Example:

```
192.168.1.234
```

Role in the system:

- Destination node in the graph.
- Identifies the most contacted targets.
- Construction of weighted graph edges (weight = BytesSent + BytesReceived).

2.6 SourcePort

SourcePort

Definition: Port number on the **source** machine.

Example:

```
8055
```

Role in the system: Unusual port activity may indicate scanning or unauthorized connection attempts. The number of unique source ports per IP is a feature for Isolation Forest.

2.7 DestinationPort

DestinationPort

Definition: Port number on the **destination** machine.

Sensitive ports examples:

- | | | |
|---------------|---------------|----------------|
| • 22 — SSH | • 80 — HTTP | • 3306 — MySQL |
| • 23 — Telnet | • 443 — HTTPS | • 3389 — RDP |

Role in the system: The number of **unique destination ports** contacted by an IP is the main indicator of a port scan.

2.8 PacketCount

PacketCount

Definition: Number of network packets transmitted in the connection.

Example:

```
827 -> 827 packets transmitted
```

Role in the system:

- $\text{Ratio PacketCount} / \text{Duration} \rightarrow$ packets per second rate.
- High value over short duration $\rightarrow$ DoS/DDoS flooding.
- Important feature for temporal spike detection.

2.9 BytesSent

BytesSent

Definition: Total amount of data **sent** by the source, in bytes.

Example:

```
142500 -> 142 KB of data sent
```

Role in the system:

- High BytesSent + low BytesReceived $\rightarrow$ data exfiltration.
- Used as edge weight in the graph.
- Feature for Isolation Forest and weighted threat score.

2.10 BytesReceived

BytesReceived

Definition: Total amount of data **received** from the destination, in bytes.

Example:

55744 -> 55.7 KB of data downloaded

Role in the system:

- Very high BytesReceived → malware download, data theft.
- Computation of the **ratio** $\frac{\text{BytesSent}}{\text{BytesReceived}}$ as an asymmetry indicator.
- Contributes to graph edge weights.

2.11 Label

Label

Definition: Connection classification (normal or attack).

Value	Meaning	Examples
BENIGN	Normal traffic	Web browsing, email
DDoS	Distributed denial of service	SYN flood, UDP flood
PortScan	Port scanning	nmap, masscan
BruteForce	Brute force attempt	SSH, RDP

Role in the system:

- Validation of alerts generated by the IDS.
- Training and evaluation of ML models.
- Stored on graph edges for visual analysis.

2.12 ☰ Features Summary Table

Feature	Description	ML Usage
Timestamp	Connection timestamp	✔ Spike detect.
Duration	Duration in seconds	✔ Isolation Forest
Protocol	TCP / UDP / ICMP	✔ Categorical feature
SourceIP	Initiating IP	✔ Graph node
DestinationIP	Destination IP	✔ Graph node
SourcePort	Source port	● Secondary feature
DestinationPort	Destination port	✔ Scan detection
PacketCount	Number of packets	✔ Isolation Forest
BytesSent	Bytes sent	✔ Edge weight
BytesReceived	Bytes received	✔ Edge weight
Label	Classification	✔ IDS validation

Table 1: Network traffic features summary

3 Development Roadmap

The project is structured into **5 progressive phases**, each consuming the outputs of the previous one. The recommended development order is strictly sequential.

✓ Phase 1 — Data Foundation [Completed]

Goal: Provide a clean, validated data foundation for all subsequent phases.

Tasks to accomplish:

1. Define the fixed CSV schema:
Timestamp, BytesSent, BytesReceived, Label
2. Write `loader.py` with function `load_csv(path) -> pd.DataFrame`
3. Parse `Timestamp` as `datetime`, cast byte columns to integers
4. Remove null or malformed rows
5. Implement `validate_schema(df)` that raises an exception on schema error
6. Normalize labels: "BENIGN", "DDoS", "PortScan", "BruteForce"

Deliverables: `loader.py`, `cleaned_data.csv`

✓ Phase 2 — Graph Construction [Completed]

Goal: Transform the tabular DataFrame into a weighted directed graph.

Tasks to accomplish:

1. Write `graph_builder.py` using NetworkX (DiGraph)
2. For each CSV row: add SrcIP/DstIP nodes + weighted edge
weight = BytesSent + BytesReceived (accumulated)
3. Store the list of Labels on each edge
4. Compute and store per node: degree, in_degree, out_degree, betweenness centrality, unique_neighbors
5. Export graph: `nx.write_gexf(G, "graph.gexf")`
6. Generate global graph statistics (density, connected components)

Deliverables: `graph_builder.py`, `graph.gexf`, `graph_stats.json`

✓ Phase 3 — Core Algorithms & ML Detection [Completed]

Goal: Extract behavioral features and detect anomalies.

Sub-task A — Per-IP feature extraction:

- Group DataFrame by SrcIP
- Compute: total BytesSent, total BytesReceived, total PacketCount, ratio $\frac{\text{BytesSent}}{\text{BytesReceived}}$, number of unique destinations, number of unique destination ports
- Merge with graph metrics (Phase 2)
- Result: a feature matrix (one row per IP)

Sub-task B — Isolation Forest:

- `sklearn.ensemble.IsolationForest(contamination=0.05)`
- Per-IP score normalized to [0, 100]
- Score near 100 → highly anomalous

Sub-task C — Temporal spike detection:

- Sort by Timestamp, 60-second sliding window
- Flag if IP exceeds 3× its own baseline within the window

Sub-task D — Weighted threat score:

$\text{ThreatScore}(IP) = 0.4 \times \text{AnomalyScore} + 0.3 \times \text{SpikeFlag} + 0.3 \times \text{DegreeCentrality}$

Deliverables: `features.py`, `ml_detector.py`, `threat_scores.json`

⚠ Phase 4 — Intrusion Detection Engine [In progress]

Goal: Merge graph + ML into a unified score and generate explainable alerts.

Tasks to accomplish:

1. Write `detection_engine.py` with a dataclass `Alert`:
`ip, score, severity, reasoning, timestamp`
2. Implement `analyze_ip(ip, features, graph, score) -> Alert`
3. Build textual reasoning programmatically:
4. Define severity thresholds:

Score	Severity	Color
< 40	LOW	Green
40–69	MEDIUM	Orange
70–89	HIGH	Red
≥ 90	CRITICAL	Purple

5. Run `analyze_ip` for all IPs in the `threat_scores` dictionary
6. Collect alerts into a list exportable as JSON/CSV

Deliverables: `detection_engine.py`, `alerts.json`, `alerts.csv`

Phase 5 — Dashboard UI [Upcoming]

Goal: Make the pipeline accessible via an interactive Streamlit interface.

app.py structure:

```
st.tabs(["Data Overview", "Graph View", "Analysis", "Detection  
Results", "Alerts"])
```

Tab details:

1. **Data Overview:** `st.dataframe` of cleaned CSV, label distribution as bar chart, byte volume over time.
2. **Graph View:** NetworkX rendering via `pyvis`, embed HTML with `st.components.v1.html`. Node color proportional to threat score (green → red).
3. **Analysis:** Feature histograms, scatter plot of `anomaly_score` vs `degree`, spike time series.
4. **Detection Results:** Table filtered by score threshold (`st.slider("Min threat score", 0, 100, 50)`), sortable by severity, CSV exportable.
5. **Alerts:** HIGH/CRITICAL alerts only, each alert inside `st.expander` with complete reasoning. `st.download_button` for CSV/report export.

❗ Important engineering rule: Keep `app.py` thin — all business logic remains in the modules from previous phases. Use `@st.cache_data` on each loading function to avoid recomputations on every user interaction.

Deliverables: `app.py`, `requirements.txt`, `README.md`

4 Conclusion

This report has presented in detail the two fundamental components of the NIDS project:

- ✓ The **11 network traffic features** — their definition, concrete examples, and precise role in the detection pipeline;
- ✓ The **5-phase roadmap** — concrete tasks to accomplish in each phase, expected deliverables, and engineering rules to follow.

The originality of this approach lies in the **combination of graph analysis** (modeling relationships between hosts) and **behavioral machine learning** (Isolation Forest), producing not only threat scores but also **explainable textual reasoning** — making the system actionable for a security analyst.

End of NIDS Technical Report
